

Splunk AI Toolkit / Ollama Performance Optimization Report

Executive Summary

This work investigated severe fixed latency in the Splunk Machine Learning Toolkit | ai command when using the custom foundationsec connection backed by a local Ollama OpenAI-compatible endpoint.

The original symptom was that a trivial prompt such as `Reply with exactly one word: OK` could take tens of seconds in Splunk even though a direct request from the Splunk host to Ollama completed in roughly 1.6 seconds. The investigation showed that the model and network were not the main bottleneck. Most delay was created by repeated Splunk-side initialization, configuration lookups, eager Python imports, and especially LiteLLM import overhead.

The final optimized path keeps the connection ACL and secret lookup, defers expensive runtime setup until real processing occurs, avoids unnecessary configuration reads for an explicitly named connection, skips an unnecessary role lookup when ACL read permissions are already wildcarded, and sends custom OpenAI-compatible requests directly with `httpx` rather than importing LiteLLM.

The largest single finding was a measured **14.594-second LiteLLM import** during execution. After routing the custom connection directly through `httpx`, the same model request measured **0.162 seconds** and the LiteLLM import disappeared from the Foundation-Sec path.

Environment Tested

- Splunk Machine Learning Toolkit under `/opt/splunk/etc/apps/Splunk_ML_Toolkit`
- Splunk Scientific Python add-on under `/opt/splunk/etc/apps/Splunk_SA_Scientific_Python_linux_x86_64`
- Scientific Python interpreter: Python 3.13.11 environment in `4_3_1`
- Custom AI Toolkit connection: `foundationsec`
- Endpoint: local Ollama OpenAI-compatible `/v1` API
- Model: `foundation-sec`
- Splunk host: `192.168.1.80`
- Ollama host: `192.168.1.190:11434`

This package is source-version-specific. It should only be installed into another environment running the same or a sufficiently compatible `Splunk_ML_Toolkit` codebase.

Baseline Investigation

Direct Ollama test

A direct call from the Splunk VM to Ollama returned quickly:

- TCP connect: approximately 0.005 s
- Time to first response: approximately 1.604 s

- Total: approximately 1.604 s

This eliminated the network path and Ollama itself as the cause of the original 25-50+ second Splunk delay.

Splunk-side baseline

A trivial SPL test was used repeatedly:

```
| makeresults
| eval prompt="Reply with exactly one word: OK"
| fields prompt
| ai prompt="{prompt}" connection="foundationsec"
```

Early measurements showed large fixed overhead in:

- Splunk search optimization / AI command setup
- AiCommanderProcessor import and constructor
- LLM configuration lookup
- MLSPLConf initialization
- LiteLLM import

One comparable early Job Inspector run showed roughly:

- `dispatch.evaluate.ai`: ~12.99 s
- `search.optimize`: ~22.75 s
- `runDuration`: ~36.6 s

Later, after the constructor/lazy-runtime work but before the final LiteLLM bypass, Job Inspector showed:

- `command.ai`: 17.34 s
- `dispatch.evaluate.ai`: 2.45 s
- `search.optimize`: 2.46 s
- `runDuration`: 22.274 s

A final end-to-end Job Inspector measurement was not captured after the direct-HTTPX change, so the report does not claim a final total `runDuration`. Component timings after the final change are documented below.

Root Causes Found

1. Eager LiteLLM import during processor import

`ai_commander/llm_base.py` originally imported LiteLLM globally. Import profiling in the Splunk Scientific Python environment showed the processor import taking about 7.9 seconds, with LiteLLM contributing the majority of that startup cost.

Moving the LiteLLM import into `litellm_post()` reduced standalone processor import from about 7.9 s to about 1.24 s. In real Splunk execution, `import_module(AiCommanderProcessor)` later measured about 0.2 s.

2. Expensive constructor ran twice

Splunk launches the AI processor once during search planning/getinfo and again during actual execution. The original `AiCommanderProcessor.__init__()` performed all expensive runtime setup both times, including connection lookup, secret retrieval, HTTP client construction, and MLSPL configuration loading.

The constructor was changed to only store lightweight state. Expensive setup was moved to `_initialize_runtime()`, called from `process()` only when data is actually processed.

Result: both processor constructors measured 0.000 sec, and the planning/getinfo invocation no longer performs runtime LLM setup.

3. LLMConfigManager loaded MLSPL configuration unnecessarily

`LLMConfigManager` created `CommonUtils(search_info)` which implicitly loaded MLSPL configuration even though LLM config ACL operations did not require allowed-domain configuration.

The constructor was changed to:

```
CommonUtils(search_info, is_mlspl_load=False)
```

This removed approximately 1.35 seconds per processor initialization at that stage of testing.

4. Named connection lookup did unnecessary legacy/default work

For an explicit `connection="foundationsec"`, the generic `get_llm_config()` path first checked legacy configuration, then checked the new KV-store, then read the default connection name even though the caller had already named the connection.

Measured costs before optimization were approximately:

- legacy lookup: 0.7 s
- new KV lookup: 0.6-0.7 s
- default connection lookup: 0.6-0.7 s
- ACL role check: 0.5-0.7 s
- secret retrieval: 0.6-0.7 s

The explicit-name path was changed to:

1. Query the new connection store first.
2. Fall back to the legacy store only if the new connection is absent.
3. Preserve ACL enforcement.
4. Skip the unrelated default-connection lookup.
5. Preserve secret retrieval.

Runtime initialization dropped from about 4.743 s to about 3.223 s after this stage.

5. MLSPLConf startup cost for retry/backoff values

`AiCommanderProcessor` spent about 1.23-1.36 seconds loading `MLSPLConf`, while `get_stanza()` itself was effectively free. In this tested path the values needed were only the standard LLM integration retry settings.

The optimized processor uses:

```
{
    "max_retries": 3,
    "backoff_factor": 2,
}
```

instead of loading MLSPL configuration at runtime.

This reduced runtime initialization from about 3.307 s to about 2.13 s in the measured run.

Important tradeoff: if another environment intentionally customizes these two values in `mlspl.conf`, this optimized file will not pick up those custom values for this path unless the code is adjusted.

6. ACL role lookup happened before wildcard ACL evaluation

`CommonUtils.is_user_eligible_by_role()` loaded the user's roles before checking whether the ACL already granted access to `"*"`.

The Foundation-Sec connection had read ACL equivalent to:

```
read: ["*"]
```

The function was reordered to evaluate owner/sharing/empty-permission/wildcard conditions before loading roles. Role retrieval still occurs when actual role matching is required.

This preserved ACL semantics while eliminating an unnecessary REST lookup. `get_llm_config()` dropped from about 1.936 s to about 1.372 s in the measured run.

7. LiteLLM import dominated actual execution

After the prior optimizations, detailed execution timing showed:

- runtime initialization: 1.507 s
- LiteLLM import: **14.594 s**
- LiteLLM completion/model call: 0.459 s

This proved that LiteLLM startup, not Ollama inference, had become the dominant remaining cost.

For the tested custom OpenAI-compatible connection, a direct `httpx` method was added. `LLMFactory` routes `is_custom=True` through `custom_openai_httpx_post()` instead of LiteLLM. The existing HTTP client is passed into `LLMFactory` so the direct method can use it.

Final measured component timings:

- runtime initialization: **1.455 s**
- direct custom HTTPX completion: **0.162 s**
- LiteLLM import on Foundation-Sec path: **none**

Files Touched

``Splunk_ML_Toolkit/bin/ai_commander/llm_base.py``

Changes:

- Changed LiteLLM from global/eager import to lazy import inside `litellm_post()`.
- Added timing instrumentation for LiteLLM import and completion.
- Added `custom_openai_httpx_post()` for direct OpenAI-compatible custom connections.

- Preserved the existing LiteLLM path for non-custom providers that still require it.

Why:

- Eager LiteLLM import was responsible for multi-second processor import latency.
- Later measurements showed a 14.594-second lazy LiteLLM import during actual execution.
- The local Ollama endpoint was already OpenAI-compatible, so LiteLLM was unnecessary for this custom connection.

`Splunk\_ML\_Toolkit/bin/ai\_commander/llm\_factory.py`

Changes:

- LLMFactory.__init__() now accepts and retains the initialized http_client.
- Custom connections are routed to custom_openai_httpx_post().
- Non-custom providers retain their existing provider-specific routing.

Why:

- The first direct-HTTPX attempt failed with 'NoneType' object has no attribute 'post' because LLMFactory did not own the HTTP client.
- Passing the same initialized client fixed the direct request path.

`Splunk\_ML\_Toolkit/bin/processors/AiCommanderProcessor.py`

Changes:

- Constructor reduced to lightweight state initialization.
- Added _initialize_runtime() and call from process().
- Runtime setup occurs only in the real execution process, not during planning/getinfo.
- Explicit named connection now checks new KV configuration first and legacy configuration second.
- Default connection lookup is skipped when an explicit connection is supplied.
- ACL and secret retrieval remain enforced.
- Runtime MLSPLConf load replaced by retry/backoff defaults.
- HTTP client is shared with LLMFactory.
- Added runtime timing instrumentation.

Why:

- Constructor work was performed twice by Splunk.
- Named connection handling did unnecessary configuration work.
- MLSPL startup added more than one second for values that were effectively static in this environment.
- LLMFactory required the HTTP client for the direct custom path.

`Splunk\_ML\_Toolkit/bin/connection\_config\_manager/llm/llm\_config\_manager.py`

Changes:

- CommonUtils is constructed with is_mlspl_load=False.
- Diagnostic timing was added around old/new/default/ACL configuration operations during the investigation.

Why:

- The manager was loading MLSPL domain configuration even though it was not needed for connection ACL operations.
- Timing instrumentation was necessary to isolate the expensive calls.

``Splunk_ML_Toolkit/bin/connection_config_manager/utils/common_utils.py``

Changes:

- Reordered ACL evaluation so owner, sharing mode, empty permissions, and wildcard * are checked before retrieving user roles.
- Role retrieval still occurs when role-specific ACL matching is actually required.

Why:

- Foundation-Sec read ACL already allowed *, so a ~0.5-0.7 second user-role REST call could never change the outcome.

``Splunk_ML_Toolkit/bin/chunked_controller.py``

Changes:

- Added timing instrumentation around processor module import, class lookup, and constructor.
- Existing load/process timing instrumentation was used to trace the execution path.

Why:

- This instrumentation proved the delay was split between module import and processor construction and later confirmed the constructor had dropped to zero after lazy-runtime initialization.

This file is diagnostic rather than required for the core optimization. It is included because it was touched and is useful when validating another environment.

``Splunk_ML_Toolkit/bin/ai.py``

Status:

- Temporarily instrumented during diagnosis, then restored to its original content.
- It is not included in the deployment payload because there is no required final functional change.

Measured Improvement Summary

Measurement	Earlier / Baseline	Optimized / Final Measured
Processor standalone import	~7.915 s	~1.238 s after lazy LiteLLM; ~0.2 s in real Splunk import timing
Processor constructor	~5-6 s	0.000 s
Search optimization	~22.75 s	2.46 s in post-lazy-runtime Job Inspector
dispatch.evaluate.ai	~12.99 s	2.45 s in post-lazy-runtime Job Inspector
Runtime initialization	4.743 s	1.455 s
<code>`get_llm_config()`</code>	~3.4 s early	1.312 s in final run
LiteLLM import during	14.594 s	0 s on custom direct-HTTPX path

execution		
Model/completion call	0.459 s through LiteLLM	0.162 s direct HTTPX
End-to-end runDuration	~36.6 s early	22.274 s before final direct-HTTPX optimization; final end-to-end not captured

Deployment Package Contents

The ZIP package contains:

- `deploy/` - replacement files in their native `Splunk_ML_Toolkit` directory structure.
- `README.md` - deployment and rollback instructions.
- `REPORT.md` - this technical report.
- `REPORT.docx` - formatted copy of this report.
- `CHANGES.patch` - unified diff against the uploaded source snapshot.
- `install.sh` - helper that backs up target files and installs the replacements while preserving owner/group/mode.
- `checksums.sha256` - integrity hashes for deployable files.
- `reference/original/` - source snapshot versions used to construct the package and review changes.

Deployment / Compatibility Warnings

6. **Version-specific source replacement.** Do not apply blindly to a different MLTK release. Compare the target files or test in a clone first.
7. **Back up all target files first.** The provided installer does this automatically.
8. **Custom-provider behavior changed.** `is_custom=True` connections now use the direct OpenAI-compatible HTTP path. This was tested with the Foundation-Sec Ollama endpoint. A custom provider that is not OpenAI chat-completions compatible may require a different route.
9. **Retry/backoff values are fixed to 3 and 2** in the optimized runtime path. If the destination environment relies on custom `mlspl.conf` values, adjust the processor before deployment.
10. **Vendor updates can overwrite these files.** Keep this package and re-evaluate after MLTK upgrades.
11. **Diagnostic logging remains in several files.** It is useful for validation but can be removed later if quieter logs are preferred.

Validation After Installation

Compile the files using the Splunk Scientific Python interpreter:

```
/opt/splunk/etc/apps/Splunk_SA_Scientific_Python_linux_x86_64/bin/linux_x86_64/4_3_1/bin/
python \
  -m py_compile \
  /opt/splunk/etc/apps/Splunk_ML_Toolkit/bin/ai_commander/llm_base.py \
  /opt/splunk/etc/apps/Splunk_ML_Toolkit/bin/ai_commander/llm_factory.py \
  /opt/splunk/etc/apps/Splunk_ML_Toolkit/bin/processors/AiCommanderProcessor.py \
  /opt/splunk/etc/apps/Splunk_ML_Toolkit/bin/connection_config_manager/llm/
llm_config_manager.py \
  /opt/splunk/etc/apps/Splunk_ML_Toolkit/bin/connection_config_manager/utils/common_utils.py
```

```
\n/opt/splunk/etc/apps/Splunk_ML_Toolkit/bin/chunked_controller.py
```

Then test:

```
| makeresults\n| eval prompt="Reply with exactly one word: OK"\n| fields prompt\n| ai prompt="{prompt}" connection="foundationsec"
```

Expected indicators in the search log:

- processor constructor approximately 0.000 sec
- runtime initialization around 1-2 seconds in the tested environment
- TIMING CUSTOM HTTPX completion
- no TIMING LITELLM import for the Foundation-Sec custom connection

Rollback

Restore the timestamped backup files created by `install.sh`, or manually restore the original files retained by your normal change-control process. Because these are Python source files executed by the external search command, new search processes will pick up restored code; if behavior appears cached, restart Splunk during a maintenance window.

Conclusion

The performance problem was not the local model. The largest delays were caused by Splunk-side setup and dependency initialization. The optimization sequence progressively removed unnecessary work until the actual custom model request became a small fraction of total time. The most important final change was bypassing LiteLLM for the OpenAI-compatible custom Ollama endpoint, eliminating a measured 14.594-second import from every Foundation-Sec execution.